

OsdagBridge Framework

Screening Task Report

Module	UI & Feature Development (Bridge Module)
Files Reviewed	steel_design_check.py, custom_load_tab.py, bridge_load_canvas.py, output_dock.py
Tasks Covered	Task 2 — Design Check Tab Task 3 — Custom Load Visualization (2D CAD)
Reference Branch	screening-task-ui (github.com/Nidhikhare12/OsdagBridge)
Standards	IRC:6 – 2017 PySide6 / Qt6
Author	Mayukh Jain
Date	April 2026

Contents

- 1 Module Overview
- 2 Task 2 — Design Check Tab
 - 2.1 SteelDesignCheckTab Class (steel_design_check.py)
 - 2.2 Equation Templates & HTML Rendering
 - 2.3 DesignCheckEngine Interface
 - 2.4 Check Execution & Parameter Wiring (_execute_checks)
 - 2.5 Status Badge System
 - 2.6 Design Check Results (Computed Values)
- 3 Task 3 — Custom Load Visualization (2D CAD)
 - 3.1 BridgeLoadCanvas Widget (bridge_load_canvas.py)
 - 3.2 CustomLoadTab Integration (custom_load_tab.py)
 - 3.3 Canvas Sync Mechanism
- 4 OutputDock Integration (output_dock.py)
- 5 Architecture Assessment
- 6 Comparison with Screening Task Requirements
- 7 Quick Reference Summary

1 Module Overview

This report analyses the four Python source files submitted as part of the FOSSEE Summer Fellowship 2026 screening task for the OsdagBridge Bridge Module. The submitted files implement two major UI features: (a) a Design Check tab inside the Steel Design dialog box, and (b) a dynamic 2D CAD cross-section visualization for

custom bridge loads. The codebase is built on PySide6/Qt6 and integrates with the existing OsdagBridge architecture.

File	Purpose	Task
steel_design_check.py	SteelDesignCheckTab — 8 design check cards with equation display,	Task 2
bridge_load_canvas.py	BridgeLoadCanvas QPainter widget — renders deck, I-beam girders, symbols	Task 3
custom_load_tab.py	CustomLoadTab — wires UI inputs to canvas, manages load list CRUD	Task 3
output_dock.py	OutputDock — config-driven dock panel; links to Steel Design dialog	Both

2 Task 2 — Design Check Tab Implementation

The screening task required populating the empty design-check cards inside the Steel Design dialog's Design Check tab with formatted equations and computed Demand-to-Capacity Ratios (DCR) for eight IRC:6 structural checks.

2.1 SteelDesignCheckTab Class (steel_design_check.py)

The class `SteelDesignCheckTab(QWidget)` is the top-level container for the Design Check tab. It uses a `StyledScrollArea` wrapping a vertical container with three main regions: a top bar (Member ID + Load Combination selectors), a 'Run Design Checks' button row, and a 2×4 grid of check cards. The constructor explicitly initialises two dictionaries before calling `super().__init__()` to avoid `AttributeError` during widget construction.

Class Instantiation

```
class SteelDesignCheckTab(QWidget):
    def __init__(self, parent=None):
        self.check_outputs = {} # key -> QTextEdit result area
        self.check_badges = {} # key -> QLabel status badge
        super().__init__(parent)
```

Attribute	Type	Purpose
check_outputs	dict[str, QTextEdit]	Readonly output area per check – displays equation and numer
check_badges	dict[str, QLabel]	Coloured badge per check – green (OK), red (FAIL), (pending)
member_combo	NoScrollComboBox	Filter by girder (populated dynamically via <code>set_girder_count()</code>)
load_combo	NoScrollComboBox	Load combination selector (Envelope, DL+LL, ...)
run_btn	QPushButton	Triggers <code>_execute_checks()</code> with hardcoded default parameters
status_lbl	QLabel	Summary e.g. '6/8 checks passed ■ Review failures'

2.2 Equation Templates & HTML Rendering

All eight equation templates are pre-rendered as HTML strings stored in the module-level dictionary `EQUATION_TEMPLATES`. Two helper functions build the HTML fragments:

- `_build_eq(*lines)` — accepts a variable list of plain strings or lists-of-items representing a horizontal expression. A list item that is a tuple (numerator, denominator) is rendered as an inline HTML fraction using a two-row table with a 1-pixel bottom border on the top cell.
- `_dcr(num, den)` — convenience wrapper that returns a list representing the $DCR = \text{num}/\text{den}$ fraction in a consistent style.

Check Key	Displayed Title	Equation Shown
flexure	Strength Limit State (Flexure)	$M_d \leq M_r = \beta_b \cdot Z_p \cdot f_y / \gamma_m$, $DCR = M_d / M_r$
shear	Strength Limit State (Shear)	$V_d \leq V_r = A_v \cdot f_y / (\sqrt{3} \cdot \gamma_m)$, $DCR = V_d / V_r$
interaction	Interaction	$M_d / M_r + V_d / V_r \leq 1$, $DCR = M_d / M_r + V_d / V_r$
ltb	Lateral Torsional Buckling	$M_d \leq M_{cr} \approx \pi^2 E I_y / L^2 \text{LTB}$, $DCR = M_d / M_{cr}$
shear_long_trans	Resistance to Long. and Transverse Shear	$V_d \leq V_{rd} = V_{rd,c} + V_{rd,s}$, $DCR = V_d / V_{rd}$
fatigue	Resistance to Fatigue	$\Delta \sigma_{allow} = \Delta \sigma C / \gamma_{mf}$, $DCR = \Delta \sigma / \Delta \sigma_{allow}$
stress	Stress Limitation	$\sigma = M_d / Z_e \leq f_y / \gamma_m$, $DCR = \sigma / (f_y / \gamma_m)$
deflection	Deflection and Crack Control	$\delta \leq L/x$ ($x=600$ default), $DCR = \delta / (L/x)$

2.3 DesignCheckEngine Interface

The actual numerical calculations are delegated to an external class `DesignCheckEngine` imported lazily inside `run_checks()` from `osdagbridge.desktop.ui.dialogs.tabs.steel_design_check_engine`. Each check is dispatched through a static method lookup table:

Check Key	DesignCheckEngine Static Method	Returns
flexure	<code>check_flexure(params)</code>	dict with 'equation_str' and 'status'
shear	<code>check_shear(params)</code>	dict with 'equation_str' and 'status'
interaction	<code>check_interaction(params)</code>	dict with 'equation_str' and 'status'
ltb	<code>check_ltb(params)</code>	dict with 'equation_str' and 'status'
shear_long_trans	<code>check_shear_long_trans(params)</code>	dict with 'equation_str' and 'status'
fatigue	<code>check_fatigue(params)</code>	dict with 'equation_str' and 'status'
stress	<code>check_stress(params)</code>	dict with 'equation_str' and 'status'
deflection	<code>check_deflection(params)</code>	dict with 'equation_str' and 'status'

2.4 Check Execution & Parameter Wiring (_execute_checks)

The private method `_execute_checks()` reads structural parameters from the widget's `_cad_state` dictionary (populated by `load_data(cad_state)`), falling back to sensible engineering defaults when no live data is available. This pattern makes the UI self-demonstrating even before a full analysis has been run.

Parameter	Source Key in cad_state	Default Value	Unit
-----------	-------------------------	---------------	------

span	span	35.0	m
fy	fy	250.0	MPa
E	E	200 000	MPa
ym	gamma_m	1.10	–
Md	moment_demand	480×10^3	N·mm
Vd	shear_demand	1 800 000	N
Zp	Zp	2 500 000	mm ³
Av	Av	18 000	mm ²
Iy	Iy	6.5×10^9	mm ⁴
Ze	Ze	2 200 000	mm ³
δ (deflection)	delta	0.048	m
L_LTB	span × 1000	35 000	mm

2.5 Status Badge System

Each check card carries a fixed-size (48 × 20 px) `QLabel` badge. The colour is driven by the string returned in the `status` field of the engine result dict:

Status Value	Badge Colour	Hex	Meaning
OK	Osdag Green	#90AF13	$DCR \leq 1.0$ — check passes
FAIL	Red	#D9534F	$DCR > 1.0$ — check fails
(pending)	Grey	#BBBBBB	Not yet computed

2.6 Design Check Results (Computed with Default Inputs)

Running `_execute_checks()` with the default parameter set (span = 35 m, fy = 250 MPa, ym = 1.10, Md = 480 kN·m, Vd = 1800 kN) produces the following results — consistent with the screenshot visible in Image 2:

Check	Equation Result	DCR	Status
Strength Limit State (Flexure)	$M_r = \beta_b \cdot Z_p \cdot f_y / \gamma_m = 568\,182 \text{ kN}\cdot\text{m}$ $M_d = 480\,000 \text{ kN}\cdot\text{m}$	0.845	OK
Strength Limit State (Shear)	$V_r = A_v \cdot f_y / (\sqrt{3} \cdot \gamma_m) = 2\,361\,887 \text{ kN}$ $V_d = 1800 \text{ kN}$	0.762	OK
Interaction	$M_d / M_r + V_d / V_r = 1.607 \leq 1$	1.607	FAIL
Lateral Torsional Buckling	$M_{cr} = \pi^2 E I_y / L^2 LTB = 104\,739 \text{ kN}\cdot\text{m}$ $DCR > 1$	4.583	FAIL
Long. & Transverse Shear	$V_{rd} = V_{rd,c} + V_{rd,s} = 1\,339\,198 \text{ kN}$	0.261	OK
Resistance to Fatigue	$\Delta \sigma_{allow} = \Delta \sigma_C / \gamma_{mf} = 61.74 \text{ MPa}$	0.842	OK

Stress Limitation	$\sigma = Md/Ze = 218.18 \text{ MPa}$ Limit = $f_y/\gamma_m = 227.27 \text{ MPa}$	0.960	OK
Deflection and Crack Control	$L/600 = 0.0583 \text{ m}$ $\delta = 0.0480 \text{ m}$	0.823	OK

Note: Interaction (1.607) and LTB (4.583) exceed the limit of 1.0, reproducing the FAIL badges visible in the screenshot. The remaining 6/8 checks pass, matching the '6/8 checks passed' ■ Review failures' status label.

3 Task 3 — Custom Load Visualization (2D CAD)

Task 3 required a dynamic 2D CAD cross-section view of the bridge showing the deck, I-beam girders, and load symbols (Point, Line, Area) that update in real time when the user changes the Load Type dropdown or position fields in the Custom Load tab.

3.1 BridgeLoadCanvas (bridge_load_canvas.py)

BridgeLoadCanvas subclasses `QWidget` and uses `QPainter` with anti-aliasing to render the entire diagram each time `paintEvent` fires. All geometry is computed dynamically from the widget's current pixel dimensions via the `_layout()` helper, so the canvas is fully responsive.

Public Setter	Signature	Effect
<code>set_load_type()</code>	<code>set_load_type(load_type: str)</code>	Switches between Point, Line, Area and triggers repaint
<code>set_span()</code>	<code>set_span(span_m: float)</code>	Updates pixel-per-metre scale factor
<code>set_position()</code>	<code>set_position(x: float)</code>	Sets the Point load longitudinal position (m)
<code>set_range()</code>	<code>set_range(x1: float, x2: float)</code>	Sets start/end positions for Line and Area loads (m)
<code>set_num_girders()</code>	<code>set_num_girders(n: int)</code>	Recomputes evenly-spaced I-beam x-positions
<code>set_span_label()</code>	<code>set_span_label(label: str)</code>	Updates the dimension-line annotation text

Paint Pipeline

The `paintEvent()` calls five drawing methods in a fixed z-order:

- `_draw_load_symbol()` – drawn BEHIND the deck so arrow tips appear to touch the deck surface
- `_draw_deck()` – filled rounded rectangle representing the deck slab
- `_draw_girders()` – three-part I-beam shape (top flange, web, bottom flange) per girder
- `_draw_labels()` – 'Deck' and 'Girder' text annotations with leader lines
- `_draw_span_dim()` – dashed horizontal dimension line with tick marks and span label
- `_draw_load_title()` – bold centred load type title above the load symbol

Load Symbol Rendering

Load Type	Visual Representation	Key Parameters
Point	Single downward arrow (shaft + filled triangle head) at position $xcx = mx + pos_x * pixel_per_meter$	
Line	Horizontal capping line from $x1$ to $x2$ with $n+1$ evenly spaced arrows	$n_arrows = \max(2, width_px // 32)$
Area	Red semi-transparent (22% alpha) dashed rectangle over loaded region with same arrow spacing	<code>QColor(204, 34, 0, 55) fill + Qt.DashLine bordering as Line load</code>

3.2 CustomLoadTab Integration (custom_load_tab.py)

`CustomLoadTab` is a `QWidget` that sits inside the Additional Inputs → Loading → Custom Load sub-tab. Its layout consists of a left column (the `BridgeLoadCanvas` + input card) and a right column (Description Box), arranged in a `QHBoxLayout`.

Key responsibilities of CustomLoadTab:

- Owns the `BridgeLoadCanvas` instance and positions it above the input card.
- Wires the Load Type combobox to a `QStackedWidget` so the Point input fields (single distance) and Line/Area fields (start/end distances) swap in place.
- Connects the distance `QLineEdit` text-changed signals to `_sync_canvas_point()` and `_sync_canvas_range()` for live canvas updates.
- Manages a `custom_load_items` list and persists entries in a `QTableWidget` with Load Case, Load Type, and Distance columns.
- Provides full CRUD: Save, Edit (re-populates input fields), and Delete with validation via `CustomMessageBox`.
- Implements `reset_defaults()` to clear all inputs and the load list.
- Exposes `sync_num_girders_from_state()` to push the girder count from `owner.cad_state` to the canvas.

3.3 Canvas Sync Mechanism

Real-time canvas updates are achieved through two private sync methods connected to `textChanged` signals:

Method	Connected To	Canvas Call
<code>_sync_canvas_point()</code>	<code>custom_point_left_input.textChanged</code>	<code>canvas.set_span(span) + canvas.set_position(val)</code>
<code>_sync_canvas_range()</code>	<code>custom_line_left_start.textChanged</code>	<code>custom_line_left_end.textChanged</code> <code>canvas.set_span(span)</code> <code>canvas.set_range(x1, x2)</code>

Both sync methods guard against invalid input with a broad `try/except`, ensuring that incomplete text entry (e.g. typing '3.' before the decimal digit) does not raise an error or crash the canvas.

4 OutputDock Integration (output_dock.py)

The `OutputDock` class is an existing framework component that was extended to accommodate the new Steel Design button. The submitted `output_dock.py` retains all original behaviour and adds the following relevant patterns:

- Config-driven field rendering via `_load_configs()` — keeps the dock's layout data-driven rather than hard-coded, consistent with the task requirement to avoid unnecessary hardcoding.
- A collapsible toggle strip (6 px green bar) with animated show/hide via `toggle_output_dock()`.
- `_create_toggle_group()` builds collapsible Superstructure / Substructure sections, each containing button rows wired to handler callbacks (e.g. opening the Steel Design dialog).
- The Steel Design 'Here' button in the Superstructure section acts as the entry point into the Task 2 dialog.

5 Architecture Assessment

The submitted code demonstrates a clear separation of concerns across three layers:

Layer	Files / Classes	Responsibility
Calculation Logic	DesignCheckEngine (external)	Pure arithmetic — equations, DCR, pass/fail logic. No UI imports.
Widget / View	SteelDesignCheckTab, BridgeLoadCanvas	Visualizes equations, results, canvas. No numeric engine code.
Tab / Controller	CustomLoadTab, SteelDesignCheckTab	load_data() Wires user input to view; manages state.

6 Comparison with Screening Task Requirements

Requirement	Status	Notes
All 8 design check equations implemented in UI	✓	All 8 checks present with HTML fraction rendering
Equations formatted (text or LaTeX-style)	✓	HTML with inline fraction tables; italic variables
DCR displayed per check	✓	DCR formula shown in eq template + numeric in result area
OK / FAIL badge per card	✓	Green (#90AF13) / Red (#D9534F) QLabel badges
'Run Design Checks' button	✓	Triggers _execute_checks() with live/default params
Member ID + Load Combination selectors	✓	NoScrollComboBox widgets; member list updates with girder count
UI layout matches screenshot (2 × 4 grid)	✓	QGridLayout with equal column stretch
Separation of UI from calculation logic	✓	Engine imported lazily; UI only displays results
Cross-section view with deck + girders	✓	QPainter I-beam girders + rounded-rect deck slab
Point load — single arrow at user position	✓	Arrow position mapped via pixel_per_meter scale
Line load — distributed arrows x1 → x2	✓	Horizontal cap line + n+1 evenly spaced arrows
Area load — shaded region + distributed arrows	✓	Dashed border + 22% alpha red fill + same arrows

Live update when inputs change	✓	textChanged signals call sync helpers instantly
Girder count driven by bridge state	✓	sync_num_girders_from_state() reads cad_state
Input validation	✓	Start > End check; empty field guard; ValueError catch
Consistent style with existing UI	✓	#90AF13 green, same fonts and spacing as base app
Elevation view (bonus)	—	Not implemented; only cross-section view present
3D OpenGL view (bonus)	—	Not implemented

7 Quick Reference Summary

Class / Function	File	Returns / Effect
SteelDesignCheckTab.__init__()	steel_design_check.py	Builds 8 check cards; initialises check_outputs / check_badges
_build_checks_grid()	steel_design_check.py	2 × 4 QGridLayout of check cards
_build_check_card(key, title)	steel_design_check.py	QFrame with title, equation label, result QTextEdit, badge
_build_eq(*lines)	steel_design_check.py	HTML string for equation template
_dcr(num, den)	steel_design_check.py	HTML list for DCR = num/den fragment
run_checks(params)	steel_design_check.py	Dispatches to DesignCheckEngine; updates outputs and badges
_execute_checks()	steel_design_check.py	Reads cad_state; assembles params dict; calls run_checks()
_update_badge(key, status)	steel_design_check.py	Sets badge colour and text
set_check_result(key, text)	steel_design_check.py	Writes text into check output area
clear_results()	steel_design_check.py	Clears all outputs; resets badges to pending
load_data(cad_state)	steel_design_check.py	Stores cad_state; calls _execute_checks()
set_girder_count(count)	steel_design_check.py	Repopulates Member ID combo
BridgeLoadCanvas.paintEvent()	bridge_load_canvas.py	Renders deck, girders, load symbol, labels, dimension line
_layout()	bridge_load_canvas.py	Returns geometry dict from current widget dimensions
_draw_point_load()	bridge_load_canvas.py	Single arrow at pos_x
_draw_line_load()	bridge_load_canvas.py	Cap line + n+1 arrows from x1 to x2
_draw_area_load()	bridge_load_canvas.py	Semi-transparent shaded rect + arrows
_arrow(cx, top_y, bot_y)	bridge_load_canvas.py	Draws shaft and filled arrowhead
CustomLoadTab._sync_canvas_poin	custom_load_tab.py	Live point-load position update to canvas
CustomLoadTab._sync_canvas_rang	custom_load_tab.py	Live line/area range update to canvas
sync_num_girders_from_state()	custom_load_tab.py	Reads cad_state.num_girders → canvas.set_num_girders()
_on_save_custom_load()	custom_load_tab.py	Validates and appends load to custom_load_items list
_on_edit_custom_load()	custom_load_tab.py	Re-populates input fields from selected row
_on_delete_custom_load()	custom_load_tab.py	Removes selected rows from load list
reset_defaults()	custom_load_tab.py	Clears all inputs and load table
OutputDock._create_toggle_group	output_dock.py	Builds collapsible Superstructure/Substructure sections
OutputDock._add_button_row()	output_dock.py	Adds label + action button row to a section

Screenshots:

